

TESTING

HelixCode Notification Testing Guide

Table of Contents

1. [Overview](#)
 2. [Running Tests](#)
 3. [Test Structure](#)
 4. [Mock Servers](#)
 5. [Writing Tests](#)
 6. [CI/CD Integration](#)
 7. [Coverage Requirements](#)
 8. [Troubleshooting](#)
-

Overview

The HelixCode notification system has comprehensive test coverage including: - **Unit Tests:** Test individual components in isolation - **Integration Tests:** Test component interactions with mock servers - **Mock Servers:** Simulate external APIs (Slack, Telegram, Discord, Email)

Test Statistics

- Total test coverage: 46.9% (notification package)
 - Mock server coverage: 100% (testutil package)
 - Total test functions: 40+
 - Lines of test code: 1,500+
-

Running Tests

All Tests

```
# Run all notification tests
go test ./internal/notification/... -v

# With coverage
go test ./internal/notification/... -v -cover
```

```
# Generate coverage report
go test ./internal/notification/... -coverprofile=coverage.out
go tool cover -html=coverage.out
```

Unit Tests Only

```
# Run specific channel tests
go test ./internal/notification -run TestSlack -v
go test ./internal/notification -run TestTelegram -v
go test ./internal/notification -run TestDiscord -v
go test ./internal/notification -run TestEmail -v
```

```
# Run mock server tests
go test ./internal/notification/testutil -v
```

Integration Tests

```
# Run all integration tests
go test ./test/integration/... -v -tags=integration

# Run specific integration test
go test ./test/integration -run TestSlackIntegration -v
    -tags=integration
go test ./test/integration -run TestDiscordIntegration -v
    -tags=integration

# With timeout (for long-running tests)
go test ./test/integration/... -v -tags=integration -timeout=5m
```

Quick Test Commands

```
# Fast: Run only unit tests
make test-unit

# Medium: Run unit + integration tests
make test-integration

# Full: Run all tests with coverage
make test-all
```

Test Structure

Directory Layout

```
HelixCode/
├── internal/
│   └── notification/
│       ├── engine.go           # Main implementation
│       ├── engine_test.go      # Unit tests
│       ├── slack_test.go       # Slack unit tests
│       ├── telegram_test.go    # Telegram unit tests
│       ├── email_test.go       # Email unit tests
│       ├── discord_test.go     # Discord unit tests
│       ├── testutil/          # Mock servers
│       │   ├── mock_servers.go
│       │   └── mock_servers_test.go
│
└── test/
    ├── integration/            # Integration tests
    │   ├── slack_integration_test.go
    │   ├── telegram_integration_test.go
    │   ├── discord_integration_test.go
    │   └── integration_test.go
```

Test Naming Conventions

- **Unit tests:** Test<Component>_<Functionality>
 - Example: TestSlackChannel_Send, TestNewEmailChannel
- **Integration tests:** Test<Component>Integration_<Scenario>
 - Example: TestSlackIntegration_WithNotificationEngine
- **Table-driven tests:** Use tests slice with name field

```
tests := []struct {
    name string
    // ... test fields
}{
    {name: "success case", ...},
    {name: "error case", ...},
}
```

Mock Servers

Overview

Mock servers simulate external APIs for testing without network calls.

Available Mock Servers

1. MockSlackServer

Simulates Slack webhook API.

Usage:

```
import "dev.helix.code/internal/notification/testutil"

func TestExample(t *testing.T) {
    mockServer := testutil.NewMockSlackServer()
    defer mockServer.Close()

    // Create channel with mock URL
    channel := notification.NewSlackChannel(
        mockServer.URL,
        "#test-channel",
        "TestBot",
    )

    // Send notification
    notification := &notification.Notification{
        Title:    "Test",
        Message:  "Test message",
        Type:     notification.NotificationTypeInfo,
    }
    err := channel.Send(context.Background(), notification)

    // Verify request was captured
    requests := mockServer.GetRequests()
    assert.Equal(t, 1, len(requests))
    assert.Equal(t, "#test-channel", requests[0].Channel)

    // Reset for next test
    mockServer.Reset()
}
```

Features: - Thread-safe request capture - Request history retrieval - Reset functionality - Request counting

2. MockTelegramServer

Simulates Telegram Bot API.

Usage:

```
mockServer := testutil.NewMockTelegramServer()  
defer mockServer.Close()
```

```
// Returns proper Telegram API response format  
// Increments message_id for each request  
// Captures chat_id, text, and parse_mode
```

Response Format:

```
{  
  "ok": true,  
  "result": {  
    "message_id": 1,  
    "chat": {"id": "123456789"},  
    "text": "Message content"  
  }  
}
```

3. MockDiscordServer

Simulates Discord webhook API.

Usage:

```
mockServer := testutil.NewMockDiscordServer()  
defer mockServer.Close()
```

```
// Returns 204 No Content (Discord standard)  
// Captures content field
```

Mock Server API

All mock servers implement these methods:

```
// GetRequests() - Retrieve all captured requests  
requests := mockServer.GetRequests()
```

```
// Reset() - Clear request history
mockServer.Reset()

// GetRequestCount() - Get number of requests
count := mockServer.GetRequestCount()

// Close() - Shut down server (always use defer)
defer mockServer.Close()
```

Writing Tests

Unit Test Template

```
package notification

import (
    "context"
    "testing"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestNewMyChannel(t *testing.T) {
    tests := []struct {
        name          string
        config         MyChannelConfig
        wantEnabled    bool
        wantError      bool
    }{
        {
            name: "valid configuration",
            config: MyChannelConfig{
                APIKey: "test-key",
                Endpoint: "https://api.example.com",
            },
            wantEnabled: true,
            wantError: false,
        },
        {
            name: "missing API key",
            config: MyChannelConfig{
                Endpoint: "https://api.example.com",
            },
        },
    }
}
```

```

        wantEnabled: false,
        wantError: false,
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        channel := NewMyChannel(tt.config)

        if tt.wantError {
            assert.Nil(t, channel)
        } else {
            assert.NotNil(t, channel)
            assert.Equal(t, tt.wantEnabled,
                channel.IsEnabled())
        }
    })
}

func TestMyChannel_Send(t *testing.T) {
    // Create mock server
    mockServer := testutil.NewMockMyServer()
    defer mockServer.Close()

    channel := NewMyChannel(mockServer.URL)

    notification := &Notification{
        Title: "Test",
        Message: "Test message",
        Type: NotificationTypeInfo,
    }

    err := channel.Send(context.Background(), notification)
    require.NoError(t, err)

    // Verify request
    requests := mockServer.GetRequests()
    assert.Equal(t, 1, len(requests))
}

```

Integration Test Template

```
//go:build integration

package integration

import (
    "context"
    "testing"
    "dev.helix.code/internal/notification"
    "dev.helix.code/internal/notification/testutil"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestMyChannelIntegration(t *testing.T) {
    mockServer := testutil.NewMockMyServer()
    defer mockServer.Close()

    // Create notification engine
    engine := notification.NewNotificationEngine()

    // Register channel
    channel := notification.NewMyChannel(mockServer.URL)
    err := engine.RegisterChannel(channel)
    require.NoError(t, err)

    // Add rule
    rule := notification.NotificationRule{
        Name:      "Test Rule",
        Condition:  "type==info",
        Channels:   []string{"mychannel"},
        Priority:   notification.NotificationPriorityMedium,
        Enabled:    true,
    }
    engine.AddRule(rule)

    // Send notification
    notification := &notification.Notification{
        Title:      "Integration Test",
        Message:    "Testing integration",
        Type:       notification.NotificationTypeInfo,
    }
```

```

err = engine.SendNotification(context.Background(),
    notification)
require.NoError(t, err)

// Verify
requests := mockServer.GetRequests()
require.Equal(t, 1, len(requests))
}

```

Best Practices

1. **Always use table-driven tests** for multiple scenarios
 2. Use **require** for **critical assertions** that should stop the test
 3. Use **assert** for **non-critical checks** that should continue
 4. **Always defer mock server cleanup**: `defer mockServer.Close()`
 5. **Reset mock servers between subtests**: `mockServer.Reset()`
 6. **Test error cases** as thoroughly as success cases
 7. Use **descriptive test names** that explain what's being tested
 8. **Add comments** for complex test logic
 9. **Test concurrent scenarios** where applicable
 10. **Test with various payload sizes** (empty, normal, large)
-

CI/CD Integration

GitHub Actions Workflow

The notification system uses GitHub Actions for automated testing.

Location: `.github/workflows/notification-tests.yml`

Triggers: - Push to main or develop branches - Pull requests to main or develop branches - Changes to notification code or tests

Jobs:

1. **unit-tests**
 - Runs all unit tests
 - Generates coverage report
 - Uploads to Codecov
2. **integration-tests**
 - Runs integration tests with `-tags=integration`
 - Timeout: 5 minutes
3. **lint**
 - Runs `golangci-lint`

- Checks code quality

4. test-summary

- Aggregates results
- Fails if any job fails

Running Locally (CI Simulation)

```
# Simulate CI pipeline locally
```

```
./scripts/ci-local.sh
```

```
# Or manually:
```

```
go test ./internal/notification/... -v -cover
```

```
go test ./test/integration/... -v -tags=integration
```

```
golangci-lint run ./internal/notification/...
```

Coverage Requirements

- **Unit Tests:** Minimum 80% coverage per package
- **Integration Tests:** All critical paths covered
- **Mock Servers:** 100% coverage (achieved)

Check coverage:

```
go test ./internal/notification -cover | grep coverage
```

Coverage Requirements

Current Coverage

- internal/notification: 46.9%
- internal/notification/testutil: 100%

Goals

- **Short-term:** 60% coverage
- **Medium-term:** 80% coverage
- **Long-term:** 90% coverage

Measuring Coverage

```
# Generate coverage profile
```

```
go test ./internal/notification/... -coverprofile=coverage.out
```

```
# View coverage by function
```

```
go tool cover -func=coverage.out

# View HTML report
go tool cover -html=coverage.out

# Check specific threshold
go test ./internal/notification/... -cover | awk
    '/coverage:/ {if ($2 < 80.0) exit 1}'
```

Troubleshooting

Common Issues

Tests Hanging

Symptom: Tests hang indefinitely

Solutions:

```
# Use timeout
go test ./test/integration/... -timeout=30s -tags=integration

# Check for goroutine leaks
go test -race ./internal/notification/...
```

Mock Server Not Responding

Symptom: connection refused errors

Solutions: - Ensure `defer server.Close()` is called - Verify you're using `server.URL` not a hardcoded URL - Check the server is started before use

```
// Correct
mockServer := testutil.NewMockSlackServer()
defer mockServer.Close()
channel := notification.NewSlackChannel(mockServer.URL, "#test",
    "bot")

// Wrong
mockServer := testutil.NewMockSlackServer()
channel := notification.NewSlackChannel("http://localhost:8080",
    "#test", "bot")
```

Integration Tests Failing

Symptom: Integration tests fail but unit tests pass

Solutions: - Ensure integration tag is used: `-tags=integration` - Check if external dependencies are required - Verify mock servers are being used (not real APIs)

Correct command

```
go test ./test/integration/... -v -tags=integration
```

Without tag, integration tests are skipped

```
go test ./test/integration/... -v
```

Import Errors

Symptom: package X is not in GOROOT

Solutions:

Update dependencies

```
go mod download
```

```
go mod tidy
```

Clear cache

```
go clean -modcache
```

```
go mod download
```

Test Data Race

Symptom: -race flag reports data races

Solutions: - Use mutex locks for shared data - Use channels for communication - Avoid shared mutable state

// Thread-safe with mutex

```
type SafeCounter struct {  
    mu    sync.Mutex  
    count int  
}
```

```
func (c *SafeCounter) Inc() {  
    c.mu.Lock()  
    defer c.mu.Unlock()  
    c.count++  
}
```

Debug Flags

Verbose output

```
go test -v
```

Show test logs

```
go test -v -args -test.v
```

Race detector

```
go test -race
```

CPU profiling

```
go test -cpuprofile=cpu.prof
```

```
go tool pprof cpu.prof
```

Memory profiling

```
go test -memprofile=mem.prof
```

```
go tool pprof mem.prof
```

Additional Resources

Documentation

- [Go Testing Package](#)
- [Testify Library](#)
- [HTTP Test Package](#)

Internal Docs

- [Notification Setup Guides](#)
- [Slack Setup](#)
- [Telegram Setup](#)
- [Email Setup](#)

Getting Help

- Check existing test files for examples
 - Review this documentation
 - Check GitHub Actions logs for CI failures
 - Run tests locally before pushing
-

Last Updated: 2025-11-04 **Test Coverage:** 46.9% (notification), 100% (testutil)

Total Tests: 40+ test functions